

Universidad Veracruzana

Licenciatura en Ingeniería de Ciberseguridad e Infraestructura de Cómputo

Programación en Red

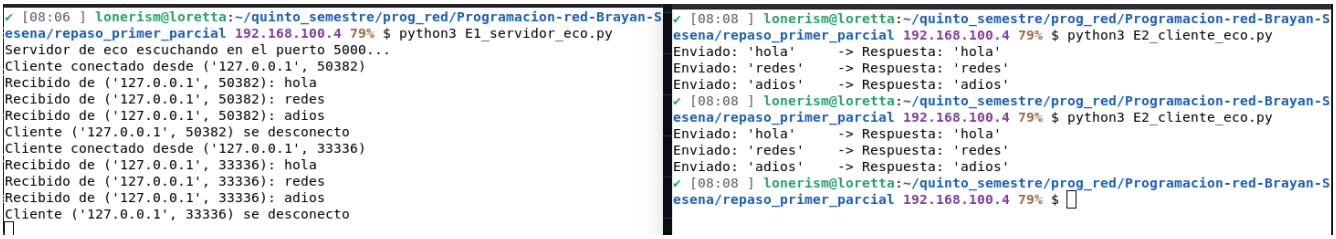
Repaso General

Brayan Usciel Seseña Hidalgo

Repaso General – Ejercicios de Programación

Ejercicio 1 · Servidor TCP de eco
Escriba el servidor de eco completo: escucha en el puerto 5000 y reenvía al cliente cada línea que recibe, con framing por salto de línea (acumule en un buffer y extraiga mensajes por \n). Debe permitir reutilizar el puerto tras reiniciarlo.
Pistas: ciclo de vida socket → bind → listen → accept; SO_REUSEADDR; el recv() vacío significa que el cliente cerró.

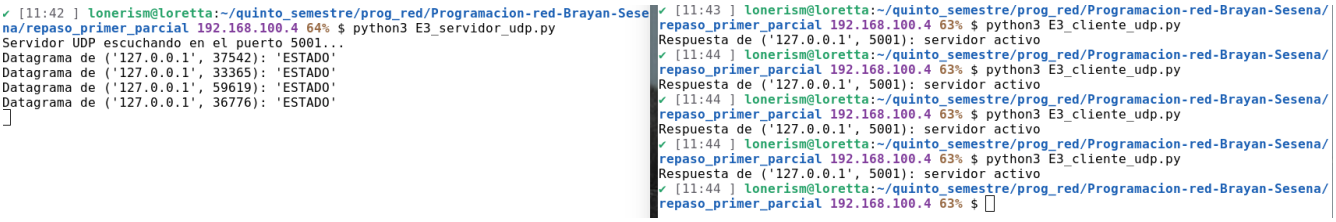
Ejercicio 2 · Cliente TCP
Escriba el cliente que se conecta al servidor del Ejercicio 1, envía tres mensajes (hola, redes, adios) y muestra la respuesta a cada uno.
Pistas: create_connection con una tupla (IP, puerto); sendall exige bytes: encode al enviar, decode al recibir.



El cliente envía tres mensajes con framing, el servidor lee los mensajes y los reenvía al cliente con framing nuevamente. El servidor identifica al socket del cliente al recibir el cliente, dado la función accept().

Cerrar la conexión de un cliente cierra su socket, pero el servidor continua escuchando más peticiones.

Ejercicio 3 · Consulta UDP
Escriba el servidor UDP (puerto 5001) que responde servidor activo cuando recibe el datagrama ESTADO, y el cliente que lo consulta esperando la respuesta a lo más 3 segundos.
Pistas: SOCK_DGRAM; en UDP no hay listen ni accept; recvfrom devuelve (datos, direccion) y se responde con sendto; el
cliente usa settimeout(3).



Ejercicio 4 · Servidor multicliente con hilos
Convierta el servidor del Ejercicio 1 en multicliente: cada conexión aceptada se atiende en su propio hilo. Compruebe con dos clientes conectados al mismo tiempo que ambos reciben su eco sin esperarse.
Pistas: función atender_cliente(conexion, direccion) como target; daemon=True; el hilo principal solo acepta.

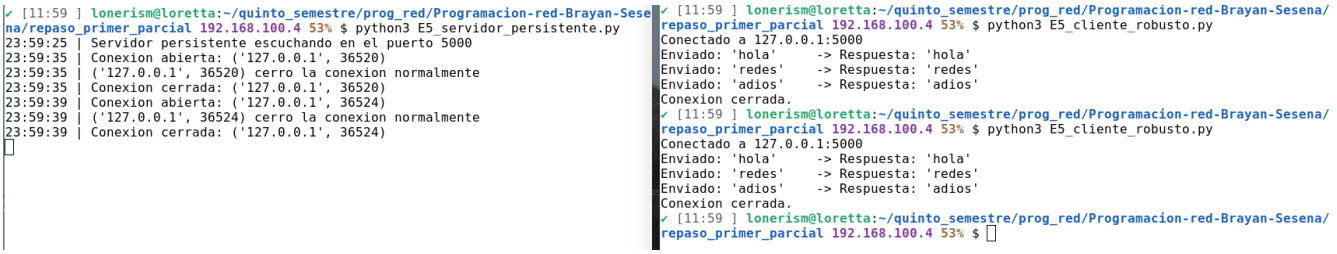


El servidor es capaz de atender a ambas solicitudes al mismo tiempo debido a que maneja cada conexión en un hilo de ejecución independiente.

Ejercicio 5 · Servidor persistente

Endurezca sus programas con try / except específicos: (a) el cliente del Ejercicio 2 debe informar con mensaje claro si el servidor está apagado, si la IP no se puede resolver o si el servidor se va a media conversación; (b) el servidor del Ejercicio 4 debe seguir atendiendo a los demás clientes aunque uno se desconecte abruptamente, y debe registrar el cierre de cada conexión con finally.

Pistas: ConnectionRefusedError y socket.gaierror al conectar; ConnectionResetError dentro del hilo de atención; el try/except del servidor vive dentro de atender_cliente.



Ejercicio 6 · Integrador: LA FILA

Programe el sistema de turnos completo. Por TCP (puerto 5000): el cliente envía TURNO>apodo\n y el servidor responde turno>N\n con números consecutivos sin repeticiones. Por UDP (puerto 5001): ante el datagrama CUANTOS el servidor responde van>N\n con el total asignado. Un hilo por cliente TCP, un hilo para el ciclo UDP, contador compartido protegido con threading. Lock y tolerancia a fallos: ningún cliente puede tumbar al servidor.

Validación: dos clientes TCP toman turno al mismo tiempo desde dos equipos y un tercero consulta por UDP, los turnos deben salir consecutivos.

• Los turnos no se repiten aunque dos clientes pidan al mismo tiempo.

